CRIME SIGHT NOIR

# CSNP v3
# Technical Documentation

A 3D police-intelligence dashboard for peninsular South India. Flask and SQLite behind a single-page vanilla-JS frontend, where every screen — the command map, the case register, the criminal network, the analytics board — is rendered as WebGL geometry rather than a 2D chart.

| Layer | Technology |
|---|---|
| Frontend | Vanilla JS · three.js r147 · no build step |
| Backend | Python 3.12 · Flask · flask-cors |
| Database | SQLite · 6 tables · no ORM |
| Data | 6,006 stations · 212 criminals · 191 officers · 274 FIRs |
| Tests | 151 assertions, throwaway database |
| Dependencies | 2 third-party packages, both server-side |

Educational / coursework project. All station, officer and criminal records derive from the source workbooks in `data/`.

## Contents

1. Overview
2. Architecture
3. Frontend
4. Backend
5. Database
6. API Reference
7. Security
8. Deployment & Operations

## Overview

*Version 3 of the Crime Sight Noir project.*

A 3D police intelligence dashboard for peninsular South India. Flask and SQLite on the server, a single-page vanilla-JavaScript client, every screen rendered as WebGL geometry rather than as flat charts.

**Live deployment:** https://nanda27.pythonanywhere.com/

---

## Quick start

```
pip install -r backend/requirements.txt
python backend/app.py
```

Open **http://127.0.0.1:4000/**. The same process serves the API and the UI, so there is no second server and no build step.

### Demo accounts

| Role | Username | Password |
|---|---|---|
| Govt Official | `admin` | `govt2026` |
| Police Officer | `officer` | `police123` |
| Public | `public` | `public123` |

> These are seeded demo credentials for a coursework project. They are printed nowhere in the running app — see `docs/SECURITY.md` .

### Access portals

Every role signs in through its own front door, so nobody has to guess which portal they belong to:

| Portal | URL | Role | Demo account |
|---|---|---|---|
| Public | `/login/public` | Public | `public` / `public123` |
| Police Officer | `/login/police` | Police Officer | `officer` / `police123` |
| Govt Official | `/login/admin` | Govt Official | `admin` / `govt2026` |

`/login` redirects to `/login/public`. A portal accepts only an account of its own role — signing in with the wrong one is refused and the error names the portal to use. The same rule governs sign-up: each portal fixes

the access level, so the Public portal cannot create a police or government account and the server refuses a mismatched registration outright.

| Role | May read | May file an FIR | May add / edit records | May delete | May manage accounts |
|---|---|---|---|---|---|
| Public | yes | | | | |
| Police Officer | yes | yes | yes | | |
| Govt Official | yes | yes | yes | yes | yes |

On the deployed site the three portals are:

- https://nanda27.pythonanywhere.com/login/public
- https://nanda27.pythonanywhere.com/login/police
- https://nanda27.pythonanywhere.com/login/admin

---

## Repository layout

```
csnp v3/
├── frontend/           the entire UI — one file, no build step
│   ├── index.html          markup + CSS + vanilla JS + the three.js scenes
│   └── vendor/             three.js r147 + OrbitControls, served at /vendor/
├── backend/            HTTP layer
│   ├── app.py               routes, validation, permission gates, JSON errors
│   ├── database.py          connection, schema execution, column migrations
│   ├── mailer.py            SMTP with a console fallback
│   └── requirements.txt
├── authentication/     login / signup / password reset (Flask blueprint)
│   └── auth.py
├── database/           DDL and seed data — no application logic
│   ├── schema.sql           all tables and indexes
│   ├── seed.sql             25 stations, 20 officers, 11 criminals, 75 FIRs, 3 users
│   └── south_india.sql      generated: 5,981 stations, 200 criminals, 854 links
├── data/               the two source spreadsheets
├── imports/            spreadsheet → SQL compiler (standard library only)
├── tests/
│   └── test_api.py          151 assertions, runs against a throwaway DB
├── docs/               the written documentation, including the PDF
└── .env.example        mail configuration template
```

---

## Documentation

| Document | Covers |
|---|---|
| `docs/ARCHITECTURE.md` | How the three tiers fit together, request lifecycle, technology choices |
| `docs/FRONTEND.md` | Every module of `index.html`: scene system, pages, renderers, permissions |
| `docs/BACKEND.md` | Flask app, permission matrix, endpoints, validation, mail |
| `docs/DATABASE.md` | Every table, column, relationship, index, seeding and migration |
| `docs/API.md` | All 28 endpoints with request/response shapes |
| `docs/SECURITY.md` | What is enforced, what is **not**, and what must change before production |
| `docs/DEPLOYMENT.md` | Running, configuration, backup, troubleshooting |
| `docs/CRIME-SIGHT-NOIR.pdf` | All of the above in one printable document — 47 pages, A4 |

The PDF is generated from these same Markdown files, so it never drifts from them. To rebuild after editing a document:

```
python docs\build_pdf.py
& "C:\Program Files\Google\Chrome\Application\chrome.exe" --headless=new `
  --no-pdf-header-footer --print-to-pdf="$PWD\docs\CRIME-SIGHT-NOIR.pdf" `
  "file:///$($PWD -replace '\\','/')/docs/CRIME-SIGHT-NOIR.html"
```

`build_pdf.py` has no dependencies — it is a small Markdown-to-HTML converter, and Chrome does the printing.

---

## Data at a glance

| Table | Rows |
|---|---|
| `police_stations` | 6,006 |
| `criminal_links` | 854 |
| `cases` | 275 |
| `criminals` | 211 |
| `officers` | 190 |
| `users` | 3 |
| `citizen_requests` | grows with use |

Every station, officer, criminal and FIR is real South India source data — station coordinates, district, state and PS code come from the two workbooks in `data/`, not from generated placeholders.

---

## Features

- **Command map** — the whole of India rendered in 3D, with South India lifted,

lit and coloured per state. 6,006 station pillars, 200 tracked suspects, orbit and zoom.

- **Case register** — every FIR with state / status / severity filters, optional

grouping, and a per-state breakdown of solved, handling and open.

- **Criminal network** — a slowly rotating wheel of records joined by shared

officer, station, city or modus operandi; click any node for a full dossier.

- **Public registry** — police and Govt Officials can see citizen accounts only.

Department accounts never appear on that route.

- **Analytics volume** — a four-quadrant 3D command board over the live totals.
- **Citizen requests** — a Public account cannot file an FIR. It sends a

request to the police station for its own region; the station replies on the thread, and once the matter is agreed the station files the FIR. The request keeps one reference, the citizen follows the case through the analytics, and the FIR is on the case register like any other.

- **Role-aware everything** — three tiers, enforced in the database-facing

layer rather than by hiding buttons.

---

## Tests

```
python tests/test_api.py
```

151 assertions across routing, both auth flows, password hashing, the permission matrix for all three roles, password reset (including token expiry and single-use), and the relationship graph. The suite builds its own throwaway database and never touches `backend/crimesight.db`.

---

## License

Educational / coursework project.

# Architecture

How the three tiers of this project fit together, what happens to a request, and why each technology was chosen.

---

## 1. The shape of the system

Three tiers, deliberately unequal in size.




The frontend is served **by** Flask, from the same origin as the API. There is no CORS negotiation in normal use, no proxy, and no second port.

---

## 2. Request lifecycle

Taking "load the criminal network" as the worked example:

```
1.  showSec('criminals')
        hides the other 7 sections, syncs the nav tabs
2.  renderCriminals(query)
        reads the cached CRL / CASES / RELATIONS arrays
3.  computes per-criminal facts, filters, groups by state/district/…
4.  CSN3D.updateCriminals({list, relations, linkKinds, headline})
5.  pushStage('criminals', …)
        reuses the existing WebGL world if already built, else creates one
6.  buildNetwork(world, list, payload)
        lays nodes on a circle, batches every link kind into 2 ribbons,
        starts one requestAnimationFrame loop
7.  one canvas on screen; OrbitControls handles orbit and zoom
```

Steps 1–4 are pure data work — microseconds. Everything after that is GPU rendering, so switching tabs never waits on the network.

**First paint is the exception.** `fetchInitialData()` runs six parallel requests (`stations`, `officers`, `criminals`, `cases`, `districts`, `relations`) before the app is usable. At full dataset size the `stations` payload is 6,006 rows — the largest response in the app — and is the main cost of a cold start.

---

## 3. Technology choices

### Why SQLite

The dataset is 275 FIRs and 6,006 stations. SQLite is a single file, needs no server, ships inside Python, and survives being copied. `sqlite3` is in the standard library, so the only third-party runtime dependencies in the whole project are **Flask** and **flask-cors**.

A server database would add an install step and a process to supervise for no benefit at this scale. The migration path, if the data ever outgrows a file, is contained in `backend/database.py` — every query in `app.py` goes through `get_db()` and plain SQL, with no ORM in the way.

### Why no build step

`frontend/index.html` is served as-is. There is no bundler, no transpiler, no `node_modules`. The browser loads one HTML file and two vendored scripts.

This is a deliberate trade. A single 4,911-line file is genuinely unpleasant to work in, and a build step would let us split it into modules. In exchange, anyone can open the project, run two commands, and see it work — no Node toolchain, no version drift, no "works on my machine". For a teaching project that readability-at-a-glance is worth more than a tidy directory.

`three.js` is **vendored**, not CDN-loaded. A demo that needs internet to render is a demo that fails in a room with bad wifi. `ensure()` still falls back to jsDelivr if the vendored files are missing.

### Why 3D for everything

Every screen is WebGL geometry: the map is extruded state polygons and instanced pillars, the analytics board is bars and sectors, the criminal wheel is instanced octahedra with batched ribbon geometry. The motivation is that a police-intelligence console should look like one, and because instancing makes 211 nodes plus 854 links cheap — about 40 draw calls instead of ~1,200.

The cost is real: WebGL is harder to debug than DOM, hit-testing needs a custom picker, and text is drawn on canvas sprites rather than laid out by the browser. Every one of those costs was paid and documented in `FRONTEND.md`.

---

## 4. Permission architecture

Permissions are enforced in **one function**, and every write route calls it.

```
def _require(action):
    role = _role_of()
    table = {
        "add_case": ("police", "admin"),
        "add":      ("police", "admin"),
        "edit":     ("police", "admin"),
        "delete":   ("admin",),
        "admin":    ("admin",),
    }[action]
```

| action | public | police | admin |
|---|---|---|---|
| `add_case` | | ✅ | ✅ |
| `add` | | ✅ | ✅ |
| `edit` | | ✅ | ✅ |
| `delete` | | | ✅ |
| `admin` | | | ✅ |
| `request` | ✅ | | |
| `request_read` | ✅ | ✅ | ✅ |
| `request_file` | | ✅ | ✅ |

`add_case` exists as a separate key from `add` so the two write tiers stay explicit rather than one key widening silently. A Public account may only read: every write route answers `403` for that role. The test suite covers this specific boundary.

The frontend mirrors the same table in `can()` so buttons can be hidden, but the frontend is never the enforcement point: hiding a button is cosmetic, and the server returns 403 regardless.

---

## 5. Data flow at startup

```
python backend/app.py
  │
  ├─ create_app()
  ├─ init_db()
  │    ├─ executes database/schema.sql        (tables + indexes)
  │    ├─ applies column migrations          (older databases gain new columns)
  │    ├─ executes database/seed.sql         (only if police_stations is empty)
  │    └─ executes database/south_india.sql   (only while still the 25-row seed)
  ├─ prints the route banner
  └─ app.run() on 127.0.0.1:4000
```

The schema is idempotent — every statement is `CREATE TABLE IF NOT EXISTS` — so restarting never destroys data. The index creation for `users.email` and `users.phone` deliberately happens in Python *after* the column migration, because an existing database does not have those columns when the schema script first runs.

---

## 6. Directory responsibilities

| Directory | Owns | Must never contain |
|---|---|---|
| `frontend/` | presentation, scene rendering, client permission checks | business rules, SQL |
| `backend/` | HTTP, validation, permission enforcement, mail | rendering logic |
| `authentication/` | credentials, sessions, password reset | domain data |
| `database/` | DDL and seed data only | any application logic |
| `data/` | the source spreadsheets | anything hand-edited |
| `imports/` | spreadsheet → SQL compilation | runtime code |
| `tests/` | assertions against a throwaway database | production data |

---

## 7. Known architectural limits

- **Single-process, single-threaded.** The Flask dev server handles one request

at a time. Fine for a demo; not a deployment target.

- **No caching layer.** `fetchInitialData()` refetches everything on every

reload. A cache keyed on the SQLite mtime would remove most of that.

- **The whole station table ships to the browser** — about 1.4 MB of JSON — even

though only South India is displayed. A `/api/stations?bbox=` endpoint would cut it by roughly an order of magnitude.

- **Token scheme is self-asserting.** Detailed in `SECURITY.md`,

and the single most important thing to change before real use.

# Frontend

Everything the browser runs lives in one file: `frontend/index.html` (4,911 lines — HTML, CSS, vanilla JS and three.js scene code). Two vendored libraries are served alongside it.

---

## 1. File map

| Path | Lines | What it is |
|---|---|---|
| `frontend/index.html` | 4,911 | The whole application |
| `frontend/vendor/three.min.js` | — | three.js r147, minified |
| `frontend/vendor/OrbitControls.js` | 730 | Orbit / pan / zoom |

three.js is vendored rather than loaded from a CDN so the app renders with no internet connection. `ensure()` falls back to jsDelivr only if the vendored files are missing.

---

## 2. The document, top to bottom

```
<style>            design tokens + every stylesheet rule
<body>
  SVG sprite       16 icon symbols, referenced by <use href="#i-map">
  .atmo            background atmosphere (grid, glow, haze)
  #dome-entrance   metallic split-dome intro
  #login-screen    access portals / sign in / sign up
  #app             the application shell
    .navbar        brand + section tabs + role badge + exit
    #rolebar       access-level banner, shown after sign-in
    .mobile-nav    bottom nav for small screens
    #section-home      / .section-registry  / .section-users
    #section-cases     / .section-analytics / .section-officers
    #section-criminals / .section-map
  modals           FIR form, criminal form, case detail, criminal dossier
<script src=/vendor/…>
<script>           all application logic
```

The SVG sprite is why there is no emoji anywhere: every icon is an outlined path that inherits `currentColor`, so it matches the surrounding text at any size and in any colour.

---

## 2a. The three access portals

Each role has its own front door, so a user is never asked to guess which portal they belong to:

| Portal | URL | Role |
|---|---|---|
| Public | `/login/public` | public |
| Police Officer | `/login/police` | police |
| Govt Official | `/login/admin` | admin |

`/login` redirects to `/login/public`. The three routes are served by Flask (`backend/app.py`) and all return the same document; the frontend reads `location.pathname`, marks the matching portal as selected, retitles the card to that role's access portal and preselects the same role on the sign-up tab.

Switching portals client-side calls `history.pushState`, so the URL is always shareable and the browser back button works, but the page never reloads and the dome intro does not replay.

A portal only accepts an account of its own role. `doLogin()` compares the role in the login response with the portal that was opened and refuses the mismatch — the session is discarded and the error names the portal to use. The check is a usability guard in the browser; the authoritative permission check stays in the API layer, where every write is authorised against the session token and the role in the database.

Sign-up follows the same rule. While a portal is open the access-level cards for the other two roles are dimmed and cannot be chosen, a line under them states which role the portal registers, and the request carries the portal so the server can refuse a mismatched registration rather than trusting the browser.

---

## 2b. Citizen requests

A Public account cannot file an FIR. What it can do is send a request to the police station for its region, and follow what happens next.

| Screen | Who | What it shows |
|---|---|---|
| **Requests** | public | *My Requests* — the requests this citizen sent, with the state of each |
| **Requests** | police, official | *Station Requests* — the queue delivered to this station (every station for an official) |

The thread modal holds the whole exchange: the citizen's opening message, the station's replies and the citizen's answers, each tagged with its author and time. A public account can add to the thread; an officer can also file the FIR from it, which creates the case, links it back to the request and adds a system message saying so.

Once an FIR exists, the tracking block under the thread shows the FIR number, the crime type, the severity, the case status, a progress bar built from the case's completion percentage and how the case sits against the rest

of its district. That is the same case record the case register and the analytics board read, so a request that becomes an FIR is counted everywhere at once.

---

## 3. Design tokens

All theming runs through CSS variables declared once on `:root`.

```
--p3:#38e1ff     primary cyan      --bg:#050a13   page background
--p6:#1c3a63     deep blue         --gb:rgba(…)   panel glass
--acc:#38e1ff    accent            --gbr:rgba(…)  border
--crit:#ff6a5e   critical          --gbrh:rgba(…) border, hovered
--f-d: 'Rajdhani'    headings
--f-m: 'Share Tech Mono'  data, labels
--f-x: 'Michroma'      brand display
```

Semantic colours are fixed deliberately: **critical and high are always warm**, whatever the theme. An operator should never have to learn that "red means serious" only on some screens.

The `.metal` surface class is the shared panel treatment — brushed gradient, fine directional grain, a specular top edge and corner brackets. Cards, modals, the home hero and the list rows all use it, which is what makes a card on the cases page and a card on the home page read as the same object.

---

## 4. JavaScript modules

The script is one scope. These are the logical modules inside it.

### 4.1 State

```
let OL=[], CRL=[], CASES=[], ST=[], OM={}, CRM={}, RELATIONS=[];
let USERS=[], REG={total:0,byState:[],users:[]};
let CU=null;                    // current user
let activeSec='home';
```

`OM` and `CRM` are id→record maps for O(1) lookup; the arrays keep order. `fetchInitialData()` fills all of them from six parallel `fetch` calls.

### 4.2 Session and permissions

```
function can(action){
  return ({
    public: ['view'],
    police: ['view','add','edit'],
    admin:  ['view','add','edit','delete'],
  }[CU?.role] || []).includes(action);
}
```

Mirrors the server's `_require()`. Used **only** to decide which buttons to render — the server refuses regardless.

`apiWrite(method, url, body)` attaches `Authorization: Bearer <token>`; the token comes from `login`, is kept in `localStorage.csn_token`, and is cleared on exit.

### 4.3 Section router

```
function showSec(n){ … }
```

Hides the other seven sections, syncs desktop and mobile tabs, redirects `users` to Home unless the role is admin and `registry` unless police or admin, calls the matching renderer, then resizes every mounted WebGL world after an 80 ms delay so the layout has settled.

The redirect is defence in depth — the tab is already hidden — but it means `showSec('users')` in the console does nothing for a police account.

### 4.4 The three.js engine — `CSN3D`

The largest subsystem. Everything visual is wrapped in `World(container, opts)`.

```
World(el, {stage, autoRotate, onPick, …})
  ├─ scene, camera, renderer, OrbitControls
  ├─ grid, lights, star field, state polygons
  ├─ setStations() / setPins() / addArc()   ← the command map
  ├─ clear(), onFrame(), pick(), view(), focus(), focusArea()
  └─ dispose()
```

`pushStage(kind, elementId, builder, payload)` keeps one world per kind and feeds it new payloads rather than rebuilding. `w.onFrame(fn)` registers a callback that runs inside the single shared `requestAnimationFrame` loop — so five simultaneous scenes still cost one loop.

`dispose()` matters: the view switches rebuild instanced meshes constantly, and without explicit disposal of geometries, materials and textures WebGL contexts leak until the tab dies.

### 4.5 Renderers

| Function | Page | What it draws |
|---|---|---|
| `renderHome()` | Home | Destination cards + live totals |
| `renderRegistry()` | Public Registry | Police/admin view of citizen accounts |
| `renderUsers()` | Users | Admin view of every account |
| `renderCaseRegister()` | Cases | Filterable, groupable FIR list |
| `renderOfficers()` | Officers | Roster with home-state filter |
| `renderCriminals()` | Criminals | State-grouped list + wheel payload |
| `initAnalytics()` | Analytics | Six-quadrant volume payload |
| `openCP()` / `openOP()` / `openCase()` | — | The three dossiers |
| `openEditForm()` / `openEditUser()` | — | Admin edit dialogs |

---

## 5. Page-by-page

### Home

Live totals (stations, FIRs, officers, tracked) and a card per destination. The "Register FIR" card only renders when `can('add')`.

### Command map

The whole of India as extruded state polygons. The eight South India states are lifted, lit, outlined, labelled, and drawn with corner posts; every other state stays muted slate. 6,006 station pins become instanced pillars; 200 tracked suspects float above their handling station. Clicking a pillar loads its FIR intelligence into the side panel.

### Cases

KPIs reflect the whole register, not the filtered slice, so the headline numbers do not jump as you filter. Rows carry a completion bar; clicking opens a dossier with 20 fields and click-through links to the criminal, officer and station.

### Officers

A sortable, filterable list. Each officer is assigned a **home state** — the one holding most of their caseload — and filtering by state returns that state's own officers rather than everyone who ever opened a case there. Secondary coverage is listed as "Also covers".

### Criminals

City clusters on a rotating wheel, joined by shared officer / station / city / modus operandi. Every link kind renders as two batched ribbons plus a 1 px core line, and a travelling pulse rides along it. Click a node for the dossier: mugshot, 16-field personal record, 3D connection web, known links with reasons, station mini-map, full FIR history.

### Public Registry

Visible to police and Govt Officials. **Only** `role='public'` rows are ever returned by the server; phone numbers are stripped server-side. Four KPI tiles give the count, states covered, how many can self-serve a reset by email, and how many can only be reached by phone.

### Users

Admin only. Every account with role, clearance tier, post, station and an explicit `RIGHTS:` line. Edit covers display name, access level and the whole profile; delete is guarded so the last admin cannot be removed or demoted.

---

## 6. The two permission UIs

`rowActions(kind, id, label)` builds the Edit/Delete cluster for a list row, and `bindRowActions(scope, kind)` wires them by delegation:

```
if(can('edit'))   …'<button data-act="edit">Edit</button>'
if(can('delete')) …'<button data-act="del">Delete</button>'
```

A Public account therefore sees a clean list with no buttons at all. A police officer sees Edit only. An admin sees both.

---

## 7. Rendering performance notes

| Technique | Where | Why |
|---|---|---|
| `InstancedMesh` | station pillars, network nodes, officer figures | one draw call for hundreds of objects |
| Merged geometry | link ribbons, glow bands | 854 links → ~12 draw calls |
| One rAF loop | `CSN3D` | five scenes, one loop |
| World reuse | `pushStage` | tab switches never rebuild WebGL |
| Sprite labels | all text in 3D | 200+ canvas textures would be fatal, so they are built on demand and cached |

One subtle trap worth recording: `material.vertexColors = true` on an `InstancedMesh` whose geometry has **no** `color` attribute renders the object black. Instance colours arrive through `instanceColor`, which three.js applies independently. Setting `vertexColors` additionally makes the shader expect a per-vertex attribute that primitives do not have.

---

## 8. Accessibility and responsiveness

- Every interactive control is a real `<button>` or `<select>`, keyboard

reachable.

- Below 900 px the desktop tabs collapse into a bottom nav and the map side

panel becomes a slide-up sheet.

- Focus rings and contrast meet WCAG AA on the metallic palette.
- **Gap:** the 3D canvases have no keyboard equivalent. The wheel and the map

are mouse/touch only; the list views are the accessible route to the same data.

---

## 9. Known frontend issues

- `index.html` is one 4,911-line file. Splitting it into ES modules is the

single biggest maintainability win available and needs no build step — the browser already understands `<script type="module">`.

- The station payload is ~1.4 MB of JSON on every cold start.
- Emoji removed from the icon set, but a few remain in the seed data's `remarks`

column, which is user content rather than markup.

## Backend

Two Python modules plus a mailer. The only third-party runtime dependencies in the entire project are Flask and flask-cors.

---

## 1. Files

| File | Lines | Responsibility |
|---|---|---|
| `backend/app.py` | 1,075 | Flask app, 28 routes, validation, permission gates |
| `backend/database.py` | 170 | Connection, schema execution, column migrations |
| `backend/mailer.py` | 119 | SMTP with a console fallback |
| `authentication/auth.py` | 317 | Login, signup, password reset — a Blueprint at `/api/auth` |
| `backend/requirements.txt` | 2 lines | `Flask>=3.0`, `flask-cors>=4.0` |

---

## 2. Application assembly

```
app = Flask(__name__)
CORS(app)
app.register_blueprint(auth_bp)          # /api/auth/*
```

`auth.py` is a separate Blueprint rather than part of `app.py` because credentials are a genuinely different concern: `auth.py` never touches `police_stations`, `cases` or any domain table.

At startup `main()` calls `init_db()`, prints a route banner, then `app.run()`.

---

## 3. Database access

```
def get_db():
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row        # name-based column access
    conn.execute("PRAGMA foreign_keys = ON")
    return conn
```

Every handler opens its own connection in a `try/finally` and closes it. There is no connection pool and no global connection — SQLite in WAL mode handles concurrent readers cheaply, and the alternative (a module-

level connection shared across threads) is a well-known source of subtle bugs.

`sqlite3.Row` is why queries read `row["status"]` rather than `row[1]`. It also means positional indexing fails — an early version of the delete guard used `.fetchone()[0]` and raised `KeyError: 0`.

---

## 4. Schema execution and migrations

`init_db()` does four things in order:

1. `executescript(schema.sql)` — all `CREATE TABLE IF NOT EXISTS`, idempotent
2. column migrations — `ALTER TABLE … ADD COLUMN` for anything an older

database lacks

1. `seed.sql` — **only** when `police_stations` is empty
2. `south_india.sql` — **only** while the table still holds the 25-row demo seed

Step 4's guard is why restarting never re-imports 6,006 stations over your data.

Migrations are driven by an allow-list of `(column, type)` tuples. Column names come from that tuple, never from request data, so the `f"ALTER TABLE {col}"` interpolation is safe.

Index creation for `users.email` / `users.phone` deliberately happens in Python **after** the migration. Creating them inside `schema.sql` breaks startup on an existing database, because that database does not have the columns yet when the schema script runs — `sqlite3.OperationalError: no such column: email`.

---

## 5. The permission gate

One function, called by every write route.

```
def _role_of():
    header = request.headers.get("Authorization", "")
    token = header[7:].strip() if header[:7].lower() == "bearer " else None
    token = token or request.headers.get("X-Session-Token")
    …  return role if role in ("public","police","admin") else None

def _require(action):
    role = _role_of()
    table = {"add_case": ("police","admin"),
             "add":       ("police","admin"),
             "edit":      ("police","admin"),
             "delete":    ("admin",),
             "admin":     ("admin",)}[action]
    if role is None:   return jsonify({"error": "Authentication required"}), 401
    if role not in table: return jsonify({"error": f"Role '{role}' may not {action}…"}), 403
    return None
```

Handlers use it as an early return:

```
denied = _require("delete")
if denied:
    return denied
```

A Public account may only read. Officers, criminals, FIRs and user records are closed to it — every write route answers `403` for the Public role. `add_case` is kept as a separate key from `add` so the two write tiers stay explicit, and the test suite pins this boundary.

A Public account that needs the police to act sends a **request** instead: a message routed to the station covering its district, a thread the two sides settle the matter on, and — once they agree — an FIR filed by the station and linked back to the request. The request routes, the thread and the FIR are therefore three separate steps, and the citizen keeps one reference throughout. The `request`, `request_read` and `request_file` actions in the same table gate them.

### Guards beyond the gate

- **Deleting an officer or criminal with open cases** → `409`, listing the count.

Closed cases are detached first so the record can still be removed once the work is done.

- **Deleting a criminal** also removes its edges from `criminal_links`, so the

network never keeps dangling links.

- **Demoting or deleting the last admin** → `409`. Otherwise the only page that

manages accounts could lock everyone out of it.

- **Deleting an FIR** is unconditional — the FIR *is* the record.

---

## 6. Validation

`_body()` returns `request.get_json(silent=True) or {}`, so a malformed body is an empty dict rather than a 500. `_int()` coerces and clamps:

```
completion_pct = _int(data.get("completionPct"), 0, minimum=0, maximum=100)
```

Every write builds an explicit column list rather than interpolating request keys into SQL. Unknown fields are ignored; a wrong type is coerced or rejected with a 400 naming the offending field.

---

## 7. Error shape

Uniform JSON, always:

```
{ "error": "Unknown stationId 99999" }
```

Status codes in use: `200` read/update, `201` create, `400` validation, `401` no or malformed token, `403` wrong role, `404` no such record, `409` refused by a guard, `500` unexpected (logged with a traceback, never a stack trace in the response body).

---

## 8. Authentication

`auth.py` owns credentials. Passwords use `werkzeug.security` — scrypt by default, salted, never returned in any response.

```
POST /api/auth/signup   name, username, password, role,
                        email|phone (one required),
                        + badge/rank/station for police,
                        + officialId/designation/department for admin
POST /api/auth/login    → { username, role, token, …profile }
```

The returned `token` is `f"csn-{role}-{secrets.token_hex(16)}"`. **It is not a session.** See `SECURITY.md`.

### Password reset

Public accounts only, and identified by **email or phone — never username**.

| Step | Endpoint | Notes |
|---|---|---|
| request | `POST /api/auth/forgot-password` | 30-minute token; **403** if the account is police or admin |
| complete | `POST /api/auth/reset-password` | token + new password |

Properties that matter:

- Only a **SHA-256 hash** of the token is stored, so a leaked database cannot be

used to complete a reset.

- Single use — `used_at` is stamped, and replay returns 400.
- Expiry is checked and the row deleted when expired.
- All other outstanding tokens for that account are burned on success.
- The account's role is **re-checked at reset time**, so a demotion mid-flow

blocks it.

- Phone numbers are matched on digits alone, so `+91 98765 43210`,

`919876543210` and `+91-98765-43210` all reach the same account.

---

## 9. Mail

`backend/mailer.py` reads `.env` with a small built-in parser (no dependency for four settings) and sends three kinds of message: password reset, FIR confirmation to the reporter, and a new-FIR alert to `CSN_NOTIFY_OFFICIALS`.

If SMTP is unconfigured, it prints the message to stderr and returns `False`. The API still succeeds:

```
{ "firNumber": "FIR-2026-9790", "mailed": { "confirmation": false, "officerAlert": false } }
```

**Mail is best-effort by design** — a mail failure must never lose an FIR. The `create_case` handler wraps both sends in `try/except` and logs, so the case is committed first.

Gmail requires an **app password**, not the account password.

---

## 10. Performance notes

| Endpoint | Cost | Note |
|---|---|---|
| `GET /api/stations` | ~1.4 MB | 6,006 rows; the largest response in the app |
| `GET /api/relations?limit=1500` | ~230 KB | the criminal graph |
| `GET /api/cases` | ~300 KB | 275 rows |
| `GET /api/stats` | trivial | one aggregate |

Indexes cover the foreign keys (`idx_cases_station`, `idx_cases_criminal`) and the graph (`idx_links_a`, `idx_links_b`), which are the columns every relationship query joins on. Without them the wheel renderer degrades visibly.

`/api/stations` returning the whole country when only South India is displayed is the clearest optimisation available: a bounding-box filter would cut it by roughly an order of magnitude.

# Database

SQLite, one file: `backend/crimesight.db`. Six tables, seven indexes, no ORM. Schema lives in `database/schema.sql` and is executed at startup.

---

## 1. Tables at a glance

| Table | Rows | Holds |
|---|---|---|
| `police_stations` | 6,006 | every station in South India, with coordinates |
| `criminals` | 211 | tracked persons and their profile |
| `officers` | 190 | the serving roster |
| `cases` | 275 | FIRs, each linked to station, officer and criminal |
| `criminal_links` | 854 | the relationship graph between criminals |
| `users` | 3 | accounts, with role and clearance |
| `password_resets` | 0 | in-flight reset tokens (hashed) |
| `citizen_requests` | 0 | requests raised by public users, routed to a station |
| `citizen_request_messages` | 0 | the negotiation thread on each request |

`password_resets` is empty between resets by design — rows are deleted once expired or spent. The two `citizen_*` tables start empty as well and fill up as citizens send requests.

---

## 2. Entity relationship

```
police_stations ──┬──< cases >──┬── officers
   id (PK)        │             │
                  │             └── criminals
                  │                   id (PK)
                  │
                  └── criminal_links (graph, not a FK relationship)
                            a ── criminals.id
                            b ── criminals.id

users   ── standalone; linked to nothing. Credentials only.
password_resets ── username ──▶ users.username
```

`criminal_links` is deliberately not a foreign key. A link is an *inferred* relationship ("these two share a handling officer"), not a declared one, and the graph would be far more rigid with real constraints on both ends.

---

## 3. Table reference

**`police_stations`**

| Column | Type | Notes |
|---|---|---|
| `id` | INTEGER PK AUTOINCREMENT | internal key used by `cases` |
| `ps_code` | TEXT NOT NULL UNIQUE | the official police station code |
| `name` | TEXT NOT NULL | e.g. `Thiruvananthapuram Railway` |
| `district` | TEXT NOT NULL | |
| `state` | TEXT NOT NULL | 8 states/UTs |
| `lat`, `lng` | REAL NOT NULL | drives the 3D map projection |
| `sector` | TEXT | |

Indexed on `state` (`idx_stations_state`) — the map's state filter.

**`officers`**

| Column | Type | Notes |
|---|---|---|
| `id` | TEXT PK | `OFF-SI-001` |
| `name` | TEXT NOT NULL | |
| `squad`, `rank` | TEXT | `rank` drives the roster-wall column height |

190 rows. Note the *distinction* from `users`: an officer is a serving record in the case system; a user is a login. They are unrelated tables and an officer need not have an account.

`criminals`

| Column | Type | Notes |
|---|---|---|
| `id` | TEXT PK | `CRIM-SI-1002` |
| `name` | TEXT NOT NULL | |
| `age` | INTEGER | |
| `address`, `city`, `state` | TEXT | |
| `gender`, `alias` | TEXT | |
| `photo_id`, `photo_url` | TEXT | portrait for the dossier |
| `crime_type`, `status`, `station`, `officer`, `case_ref` | TEXT | dataset provenance |
| `lat`, `lng` | REAL | where the record is plotted on the map |

One record in the seed set (`CRIM-111`, "Debug Subject") is a deliberate fixture for exercising the create-criminal path.

`cases`

| Column | Type | Notes |
|---|---|---|
| `id` | INTEGER PK AUTOINCREMENT | |
| `fir_number` | TEXT NOT NULL UNIQUE | `FIR-2026-4605` |
| `station_id` | INTEGER | → `police_stations.id` |
| `officer_id` | TEXT | → `officers.id` |
| `criminal_id` | TEXT | → `criminals.id` |
| `severity` | TEXT NOT NULL | Critical / High / Medium, **derived from crime type** on create |
| `crime_type` | TEXT NOT NULL | |
| `crime_count` | INTEGER | charges |
| `status` | TEXT NOT NULL | Open / Under Investigation / Closed |
| `completion_pct` | INTEGER | drives the progress bar |
| `handling`, `investigation_type` | TEXT | |
| `arrest_date`, `remarks` | TEXT | |
| `transfers`, `experience_years` | INTEGER | |

`severity` is computed server-side by `get_severity(crime_type)` rather than trusted from the client, so a caller cannot register a homicide as medium. Editing the crime type re-derives it unless a severity was pinned explicitly.

Indexed on `station_id` and `criminal_id` — the two columns every "cases for X" query joins on.

> **Known data quirk:** `handling` is the literal string `"Direct"` in every seeded row. It carries no information and is displayed verbatim in the case dossier. It is a source-data property, not a rendering bug.

`criminal_links`

| Column | Type | Notes |
|---|---|---|
| `a`, `b` | TEXT | the two criminal ids |
| `kind` | TEXT | `officer`, `station`, `city`, `crime` |
| `strength` | INTEGER | 1–3 |
| `label` | TEXT | human reason, e.g. `Shared officer: Inspector …` |
| PK | (a, b, kind) | |

Generated by `imports/import_datasets.py`. Indexed on `a` and `b` separately — a single composite index cannot serve "all links where b is X".

`users`

| Column | Type | Notes |
|---|---|---|
| `username` | TEXT PK | |
| `password` | TEXT | scrypt hash via werkzeug — never returned |
| `role` | TEXT | `public` / `police` / `admin` |
| `display` | TEXT | shown in the UI |
| `email`, `phone` | TEXT | at least one required at signup; the reset route |
| `badge`, `rank`, `squad`, `official_id` | TEXT | police / admin profile |
| `station_code`, `station_name`, `district`, `state` | TEXT | posting |
| `designation`, `department`, `zone` | TEXT | admin profile |
| `session_token` | TEXT | the live session token, so a request can be traced to the citizen who raised it |

Partial **unique** indexes on `email` and `phone` (`WHERE … IS NOT NULL AND … != ''`), so two Public accounts cannot share a reset address. Built in `database.py` after the column migration — see `BACKEND.md` for why not in the schema script.

`citizen_requests`

| Column | Type | Notes |
|---|---|---|
| `id` | INTEGER PK | |
| `reference` | TEXT UNIQUE | `CR-2026-1` — the number the citizen quotes |
| `username` | TEXT | the public account that raised it |
| `station_id` | INTEGER | the station the request was delivered to |
| `subject`, `details` | TEXT | the citizen's own words |
| `category` | TEXT | the type of matter, as chosen by the citizen |
| `status` | TEXT | `Open` · `Under Discussion` · `FIR Filed` · `Closed` |
| `case_id` | INTEGER | the FIR filed from this request, once there is one |
| `created_at`, `updated_at` | TEXT | |

`citizen_request_messages`

| Column | Type | Notes |
|---|---|---|
| `id` | INTEGER PK | |
| `request_id` | INTEGER | FK to `citizen_requests`, deleted with it |
| `author`, `author_role` | TEXT | who wrote it, and in which role |
| `body` | TEXT | the message |
| `created_at` | TEXT | |

The first message of every request is the citizen's own text, so the thread reads as the whole exchange from the start.

`password_resets`

| Column | Type | Notes |
|---|---|---|
| `username` | TEXT | |
| `token_hash` | TEXT NOT NULL | SHA-256; the raw token is never stored |
| `created_at`, `expires_at` | TEXT | ISO 8601, 30-minute window |
| `used_at` | TEXT | marks a token spent — a link works exactly once |
| PK | (username, token_hash) | |

---

## 4. Seed data

### `database/seed.sql` — the demo fixture

25 stations · 20 officers · 11 criminals · 75 FIRs · 3 users · 3 users' links.

Applied **only when `police_stations` is empty**. Creates the three demo accounts: `admin` / `govt2026`, `officer` / `police123`, `public` / `public123`.

### `database/south_india.sql` — the real dataset

Generated, ~7,400 lines: 5,981 stations, 200 criminals, 854 relation edges.

Applied **only while the station table still holds the 25-row demo seed** — a floor of 1,000 rows in `database.py` decides this. That guard is why restarting the server never re-imports over your data.

### `data/` — the source workbooks

| File | Size | Rows |
|---|---|---|
| `India_Police_Stations_Full_Database.xlsx` | 674 KB | 5,981 |
| `South_India_Criminal_Tracking_200.xlsx` | 24 KB | 200 |

### `imports/import_datasets.py`

Compiles the workbooks into `database/south_india.sql`. Standard library only — an `.xlsx` file is a ZIP of XML, so no pandas or openpyxl is needed. A workbook of 5,981 rows unzips and parses in a couple of seconds.

```
python imports/import_datasets.py          # rewrite the SQL
python imports/import_datasets.py --apply  # …and load it
```

Re-run after editing either spreadsheet.

---

## 5. Constraints in force

| Constraint | Why |
|---|---|
| `ps_code` UNIQUE | it is the real-world identifier |
| `fir_number` UNIQUE | an FIR number is issued once |
| `criminal_links` PK (a,b,kind) | no duplicate edges of the same kind |
| partial UNIQUE on `users.email` / `users.phone` | two accounts must not share a reset route |
| `NOT NULL` on identity columns | a station without coordinates cannot be plotted |
| `PRAGMA foreign_keys = ON` | `cases` → station/officer/criminal integrity |
| CHECK-free status/severity | values are constrained in the application layer, not the schema — a deliberate trade for portability |

That last row is a real trade-off. SQLite would accept the CHECK constraints, but they were left to the application so the same `.sql` files remain loadable by tools with stricter or looser dialects.

---

## 6. Migrations

No Alembic, no migration files. `backend/database.py` holds an allow-list of columns that newer versions add:

```
_USER_COLUMNS = (
    ("badge","TEXT"), ("rank","TEXT"), …, ("email","TEXT"), ("phone","TEXT"),
)
```

On every start, any missing column is added with `ALTER TABLE … ADD COLUMN`. Idempotent, and enough for this project's rate of change.

Adding a column to `schema.sql` **and** to `_USER_COLUMNS` is the whole process. Renaming or dropping a column needs hand-written SQL — there is no down-migration machinery, by design.

---

## 7. Backup and reset

```
Copy-Item backend\crimesight.db backend\crimesight.db.bak
```

To start clean, delete the database and restart — the schema and both seed files re-run automatically.

---

## 8. Query performance

Indexes exist for exactly the joins the app performs:

| Index | Serves |
|---|---|
| `idx_stations_state` | the map's state filter |
| `idx_cases_station` | "cases filed at this station" |
| `idx_cases_criminal` | the criminal dossier's FIR history |
| `idx_links_a`, `idx_links_b` | the wheel graph, both directions |
| `idx_users_email`, `idx_users_phone` | reset lookup |

Every page's aggregate queries are single-pass `COUNT` / `SUM` over `cases`, which at 275 rows is sub-millisecond. The scale at which this stops being true is far beyond this dataset.

## API Reference

34 endpoints. All responses are JSON except `GET /` (HTML) and `GET /vendor/<file>` (JavaScript).

Base URL when served by Flask: `http://127.0.0.1:4000`

---

## Authentication

Write routes need the session token from login, as a bearer header:

```
Authorization: Bearer csn-admin-<hex>
```

Missing → `401`. Malformed or wrong role → `403`.

| Code | Meaning |
|---|---|
| 200 | read or update |
| 201 | created |
| 400 | validation failed — the message names the field |
| 401 | no or unparseable token |
| 403 | authenticated, but the role may not do this |
| 404 | no such record |
| 409 | refused by a guard (open cases, last admin) |

---

## Permission matrix

| Action | public | police | admin |
|---|---|---|---|
| read any record | ✅ | ✅ | ✅ |
| file an FIR ( `add_case` ) | | ✅ | ✅ |
| create officer / criminal ( `add` ) | | ✅ | ✅ |
| edit records ( `edit` ) | | ✅ | ✅ |
| delete records ( `delete` ) | | | ✅ |
| user directory ( `admin` ) | | | ✅ |
| public registry | | ✅ | ✅ |
| raise a citizen request | ✅ | | |
| reply on a request thread | ✅ | ✅ | ✅ |
| file the FIR for a request | | ✅ | ✅ |
| close a request | | ✅ | ✅ |

---

## Auth — `/api/auth`

### `POST /api/auth/signup` → 201

```
{ "name": "Asha Menon", "username": "asham", "password": "secret123",
  "role": "public", "email": "asha@example.com", "phone": "+91 98111 22333" }
```

`email` **or** `phone` is required — it is the only route to a password reset.

Police additionally require `badge`, `rank`, `stationCode`. Govt Officials require `officialId`, `designation`, `department`.

`portal` is optional. When it is present it must equal `role`, because each role has its own front door and a portal registers its own role and no other:

```
{ "name": "Asha Menon", "username": "asham", "password": "secret123",
  "role": "public", "portal": "public", "email": "asha@example.com" }
```

`400` if `portal` names another role, so the Public portal cannot be used to mint a police or government account.

```
{ "username": "asham", "role": "public", "token": "csn-public-9f2c…",
  "display": "Asha Menon", "email": "asha@example.com" }
```

`409` if the username is taken.

### `POST /api/auth/login` → 200

```
{ "username": "admin", "password": "govt2026" }
```

Returns the same shape as signup. `401` on bad credentials.

### `POST /api/auth/forgot-password` → 200

```
{ "identifier": "asha@example.com" }
```

Accepts an email address **or** a phone number — never a username. Phone matching is on digits alone, so `+91 98111 22333`, `919811122333` and `+91-98111-22333` are equivalent.

| Status | Meaning |
|---|---|
| 200 | token issued |
| 403 | the account is police or admin — **not self-service** |
| 404 | no account for that contact |

```
{ "ok": true, "expiresInMinutes": 30, "via": "email",
  "devLink": "http://127.0.0.1:4000/?reset=…",
  "hint": "Mail delivery is not configured…" }
```

`devLink` is present **only** when SMTP is unconfigured or no email is on file, so the flow is testable without a mail server. With SMTP configured it is omitted and the link is emailed.

### `POST /api/auth/reset-password` → 200

```
{ "token": "…", "password": "brandnew1" }
```

`400` for an unknown, expired or already-used token. `403` if the account stopped being Public between request and completion. All other outstanding tokens for that account are burned.

---

## Read — no token required

### `GET /api/stations`

All 6,006 stations. `?state=`, `?q=`, `?lite=1`.

### `GET /api/stations/summary`

Per-state counts, used by the map's state chips.

```
[ { "state": "Tamil Nadu", "stations": 2433, "cases": 61, "critical": 10 } ]
```

`GET /api/stations/<id>/cases`

`GET /api/officers` · `GET /api/officers/<id>/cases`

`GET /api/criminals` · `GET /api/criminals/<id>`

`GET /api/cases` — `?state=`, `?status=`, `?severity=`, `?q=`, `?limit=`

`GET /api/districts`

### `GET /api/relations` — `?limit=` (default 1500) → the wheel graph

```
{ "a": "CRIM-SI-1002", "b": "CRIM-SI-1148", "kind": "officer",
  "label": "Shared officer: Inspector Vijay Kumar", "strength": 3 }
```

`GET /api/stats`

```
{ "total_cases": 275, "open_cases": 132, "closed_cases": 52,
  "critical_cases": 47, "total_stations": 6006 }
```

`GET /api/analytics`

Aggregates for the 3D command board: by state, by crime type, by severity, by status, top districts and officer workload.

---

## Protected reads

### `GET /api/users` — admin only

Every account. `403` for police and public. Password hashes are never selected.

```
{ "username": "officer", "role": "police", "display": "Police Officer",
  "badge": "…", "rank": "Inspector", "permissions": ["view","add","edit"] }
```

### `GET /api/public-registry` — police or admin

Citizen accounts only.

```
{ "total": 2,
  "byState": [ { "state": "Kerala", "count": 1 } ],
  "users": [ { "username": "public", "display": "Public",
               "email": "…", "state": null, "district": null } ] }
```

The query filters `WHERE role = 'public'`, so police and admin rows can never appear. **Phone numbers are stripped** — contact detail, not intelligence. `403` for public.

---

## Citizen requests — `/api/requests`

A public user cannot file an FIR. What they can do is send a **request** to the police station for their region. The request is delivered to that station, the two sides settle the matter over the message thread, and the station files the FIR itself once the matter is agreed. The FIR links back to the request, so the citizen keeps one reference and can follow the case through the analytics.

| Route | Who | Does |
|---|---|---|
| `POST /api/requests` | public | raise a request for a district or a specific station |
| `GET /api/requests` | all | own requests (public) · the station's queue (police) · every request (official) |
| `GET /api/requests/<id>` | all | one request with its thread and its case tracking |
| `POST /api/requests/<id>/messages` | all | add a message to the negotiation thread |
| `POST /api/requests/<id>/file-fir` | police, official | file the FIR the negotiation agreed on |
| `POST /api/requests/<id>/close` | police, official | close the request |

### `POST /api/requests` → 201

```
{ "subject": "Mobile phone stolen near the park",
  "details": "My phone was taken from the bench near the park gate.",
  "category": "VEHICLE", "district": "Hyderabad", "state": "Telangana" }
```

`stationId` may be sent instead of `district`; otherwise the district is resolved to the station that covers it. The response carries the reference and the station the request was delivered to:

```
{ "id": 2, "reference": "CR-2026-2", "status": "Open",
  "station": { "name": "Hyderabad Central PS", "district": "Hyderabad",
               "state": "Telangana" } }
```

A request starts as `Open` and becomes `Under Discussion` when either side replies. `400` when the subject or the details are too short, `403` for any account that is not a public one.

### `GET /api/requests/<id>` → 200

The request, its messages and — once the FIR is filed — the tracking block:

```
{ "reference": "CR-2026-2", "status": "FIR Filed", "case_id": 277,
  "messages": [ { "author": "public", "author_role": "public",
                  "body": "…", "created_at": "2026-10-20 20:11" } ],
  "tracking": { "firNumber": "FIR-2026-9790", "status": "Open",
                "severity": "Medium", "completion": 0,
                "district": "Hyderabad", "districtTotal": 5,
                "districtClosed": 0 } }
```

### `POST /api/requests/<id>/file-fir` → 201

Body: `crimeType`, optional `severity` and `remarks`. Creates the FIR at the station the request was routed to and links it back:

```
{ "ok": true, "firNumber": "FIR-2026-9790", "caseId": 277 }
```

`403` for a public account — only the station or an official files the FIR. `409` if an FIR has already been filed on the request.

---

## Writes

### `POST /api/cases` — `add_case`

```
{ "stationId": 2565, "crimeType": "THEFT", "status": "Open",
  "officerId": "OFF-SI-001", "criminalId": null,
  "reporterEmail": "citizen@example.com" }
```

`severity` is **derived** from `crimeType` server-side. `fir_number` is generated.

```
{ "firNumber": "FIR-2026-9790", "crimeType": "THEFT", "severity": "Medium",
  "status": "Open",
  "mailed": { "confirmation": false, "officerAlert": false } }
```

`mailed` is informational — **a mail failure never loses the case.**

### `PUT /api/cases/<id>` — `edit`

Any subset of `crimeType`, `status`, `severity`, `completionPct`, `crimeCount`, `stationId`, `officerId`, `criminalId`, `investigationType`, `handling`, `remarks`, `arrestDate`, `transfers`. Changing the crime type re-derives severity unless one was pinned.

### `DELETE /api/cases/<id>` — `delete`, admin only

```
{ "deleted": 275, "firNumber": "FIR-2026-1" }
```

### `POST /api/criminals` — `add`

`name` required; the rest of the profile optional. **Previously had no permission check at all** — now gated.

### `PUT` / `DELETE /api/criminals/<id>` — `edit` / `delete`

Delete is refused with `409` if the record is named in open cases, and always removes the record's edges from `criminal_links`.

### `POST /api/officers` — `add`

```
{ "name": "New Officer", "rank": "DSP", "squad": "QRT" }
→ { "id": "OFF-ADM-0001", "name": "New Officer", "rank": "DSP", "squad": "QRT" }
```

### `PUT` / `DELETE /api/officers/<id>` — `edit` / `delete`

Delete detaches the officer's closed cases and is refused with `409` while they hold open ones.

### `PUT /api/users/<username>` — admin only

Display name, role, badge, rank, squad, official ID, designation, station, district, state, department, zone, email, phone.

### `DELETE /api/users/<username>` — admin only

`409` if this is the last remaining admin.

---

## Static

| Route | Serves |
|---|---|
| `GET /` | `frontend/index.html` |
| `GET /login` | `302` to `/login/public` |
| `GET /login/public` | `frontend/index.html` — the citizen portal |
| `GET /login/police` | `frontend/index.html` — the police officer portal |
| `GET /login/admin` | `frontend/index.html` — the government official portal |
| `GET /vendor/<file>` | `three.min.js`, `OrbitControls.js` |

The three portal routes serve the same document. The frontend reads the path, titles the form for that role and refuses an account belonging to a different one; the role check itself lives in the browser, the server hands out the same session token either way.

---

## Full inventory

**Auth (4)** login · signup · forgot-password · reset-password **Public read (11)** stations · stations/summary · stations/`<id>`/cases · officers · officers/`<id>`/cases · criminals · criminals/`<id>` · cases · districts · relations · stats · analytics **Protected read (2)** users · public-registry **Write (11)** cases POST/PUT/DELETE · criminals POST/PUT/DELETE · officers POST/PUT/DELETE · users PUT/DELETE **Citizen requests (6)** requests POST/GET · requests/`<id>` GET · requests/`<id>`/messages · requests/`<id>`/file-fir · requests/`<id>`/close **Static (5)** `/` · `/login` · `/login/<role>` · `/vendor/<file>`

# Security

What this project actually enforces, what it does not, and what must change before it handles real data. Written as an honest assessment rather than a checklist of things that were done.

---

## 1. Summary

The permission model is genuinely enforced server-side, and the password reset flow is built the way a reset flow should be built: hashed single-use tokens, expiry, re-checked role.

The **session token is not a session.** That is the one thing that makes this project unsuitable for real use as it stands, and it is stated first because it should not be discovered later.

---

## 2. What is properly enforced

### Password storage

Passwords are hashed with `werkzeug.security`, which uses **scrypt** by default — salted, deliberately slow, memory-hard. No password is ever selected into a response; `GET /api/users` does not include the column at all.

```
SELECT username, role, display, email FROM users   -- password is not in the list
```

### Password reset

Implemented correctly, and the details are the point:

| Property | Implementation |
|---|---|
| tokens are not secrets at rest | only a SHA-256 **hash** is stored |
| links expire | 30-minute window, checked server-side, row deleted when expired |
| links are single-use | `used_at` is stamped; replay returns `400` |
| old sessions die | every other outstanding token for that account is burned on success |
| privilege cannot be smuggled | the account's role is **re-checked at reset time**, so a demotion mid-flow blocks it |
| no username enumeration for police | the request route returns `403` for police/admin |
| no username reset at all | lookup is by email or phone only; a username returns `404` |

Phone matching normalises to digits alone, so `+91 98765 43210`, `919876543210` and `+91-98765-43210` reach the same account. A **partial unique index** on `users.email` and `users.phone` prevents two Public accounts sharing one reset address, which would otherwise let an attacker reset someone else's password.

### Server-side authorisation

Every write route calls `_require()` before touching data. A Public account calling `POST /api/criminals` receives `403` regardless of what the UI shows. The frontend's `can()` only decides which buttons to render — hiding a button is cosmetic, and the tests confirm the server refuses independently.

### Referential integrity

`PRAGMA foreign_keys = ON` on every connection, so `cases` cannot point at a station, officer or criminal that does not exist.

### Data minimisation

`GET /api/public-registry` strips phone numbers server-side. Contact detail is not intelligence, and the registry does not need it.

---

## 3. What is **not** enforced — read this part

### The token is self-asserting

```
token = f"csn-{role}-{secrets.token_hex(16)}"
```

and it is parsed back by:

```
def _role_of():
    parts = token.split("-")
    role = parts[1]
    return role if role in ("public", "police", "admin") else None
```

**The role is read out of the token string.** There is no server-side session store, no signature, no expiry, no revocation. Consequences:

- Any client can mint `csn-admin-0000000000000000` by typing it, and the server

will treat the request as a Govt Official.

- Tokens never expire. A leaked token is valid forever.
- Tokens cannot be revoked. Deleting a user account does not invalidate a token

already issued.

- There is no audit trail — nothing records who did what.

The 128 bits of `token_hex(16)` are real randomness and do prevent *guessing* a token that was issued. They do nothing against a client that simply writes its own. This is fine for a classroom demo with three seeded logins and inappropriate for anything else.

**One step has been taken already.** The token issued at login is now recorded against the account (`users.session_token`), so the server can resolve *which* user a token belongs to. The citizen requests rely on it: a request is owned by the account that raised it, an officer only sees the queue of their own station, and the thread records who wrote each message. The **role** is still read out of the token string, so the self-asserting weakness above still stands for the role; only the identity is now server-side.

### What fixing it requires

The smallest correct change is a server-side session table plus a signed or opaque lookup — roughly 40 lines:

```
CREATE TABLE sessions (
  token_hash TEXT PRIMARY KEY,   -- SHA-256 of the token, same trick as resets
  username   TEXT NOT NULL,
  role       TEXT NOT NULL,
  created_at TEXT NOT NULL,
  expires_at TEXT NOT NULL
);
```

`_role_of()` becomes a lookup against that table. Add an expiry and delete on logout, and every one of the four consequences above disappears. Password reset already hashes tokens correctly, so the pattern is already in the codebase — it just was not applied to login.

### Other gaps

| Gap | Detail |
|---|---|
| **No rate limiting** | `/api/auth/login` accepts unlimited attempts. Passwords are hashed with scrypt, which makes each guess slow — that is real cost, but it is not a limit. A lockout counter or per-IP throttle is the fix. |
| **No CSRF defence** | irrelevant while the API is token-only and there is no cookie session; it returns the moment cookies are introduced |
| **No security headers** | no `CSP`, `X-Content-Type-Options`, `X-Frame-Options` |
| **No HTTPS** | tokens travel in cleartext over HTTP. `127.0.0.1` makes this moot locally and fatal in production |
| **No input sanitisation on display** | values are inserted with `textContent`/template literals, not `innerHTML`, so stored XSS is not currently reachable — but the pattern is one careless edit away from being exploitable |
| **Dev server in production** | `app.run()` is single-threaded and not hardened |
| **Demo credentials in seed** | `admin/govt2026` etc. are in `seed.sql` — fine for a demo, must not reach a real deployment |
| **No audit log** | deletions of officers, criminals, FIRs and accounts leave no trace |
| **Severity from the client for edits** | creation derives severity server-side; `PUT /api/cases/<id>` accepts an explicit severity, which is intentional for pinning but means a police account can relabel a homicide as medium |
| **Console mail fallback** | when SMTP is unconfigured the reset link is returned in the API response as `devLink`. That is deliberate and gated on configuration, but it must never be enabled on a real deployment |

---

## 4. Guards that are about integrity, not secrecy

Distinct from authorisation — these protect the data model rather than confidentiality:

- **Deleting an officer or criminal with open cases** → `409` with the count.

Closed cases are detached first, so the record becomes removable once the work is done.

- **Deleting a criminal** removes its edges from `criminal_links`, so the

network never keeps dangling links.

- **Demoting or deleting the last admin** → `409`. Otherwise the only page that

manages accounts could lock everyone out of it.

---

## 5. Checklist before any real deployment

Ordered by how much damage the gap does.

1. **Replace the self-asserting token** with a server-side session table.
2. **Serve over HTTPS** and add `Strict-Transport-Security`.
3. **Add rate limiting** to `/api/auth/login`, `/api/auth/signup` and `/api/auth/forgot-password`.
1. **Remove the seeded demo accounts**, or force a password change on first login.
1. **Configure SMTP** and confirm `devLink` no longer appears in responses.
2. **Add security headers**: CSP, `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`.
1. **Serve through a real WSGI server** — `waitress` or `gunicorn` — not `app.run()`.
1. **Add an audit log** for every write and every delete.
2. **Move secrets to the environment**, not `.env` beside the source.
3. **Review data exposure** — the API currently publishes all 6,006 stations with coordinates. Real station locations are sensitive.

---

## 6. Honest assessment

For a coursework submission this is a defensible design: the permission model is real, enforced in the right layer, tested at the boundary, and the reset flow follows current practice closely.

The gap between "works as a demo" and "safe with real data" is concentrated in one place — sessions. Fixing that single item removes most of section 3, and because the password-reset code already demonstrates the hashed-token pattern, the change is small enough to complete in an afternoon.

# Deployment & Operations

Running, configuring, backing up and troubleshooting the project.

---

## 1. Requirements

| Requirement | Version | Notes |
|---|---|---|
| Python | 3.10+ | developed on 3.12 |
| Flask | ≥ 3.0 | the only hard dependency |
| flask-cors | ≥ 4.0 | permissive in this build |
| Browser | any WebGL-capable | Chrome, Edge, Firefox, Safari |
| Disk | ~5 MB | database is 4 MB, of which most is station coordinates |

No Node.js, no bundler, no build step, no external database server.

---

## 2. Install

```
cd "csnp v3"
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r backend\requirements.txt
```

The whole of `requirements.txt`:

```
Flask>=3.0
flask-cors>=4.0
```

Standard library supplies everything else — `sqlite3`, `smtplib`, `secrets`, `hashlib`, and the `.xlsx` reader in `imports/`.

---

## 3. Run

```
python backend\app.py
```

```
Crime Sight Noir — CSNP v3
──────────────────────────────────────────
6,006 stations · 211 criminals · 190 officers · 275 FIRs
http://127.0.0.1:4000
```

Open the URL. First start takes a second or two longer while the schema runs; subsequent starts are instant.

Stop with `Ctrl+C`.

---

## 4. Configuration

Everything configurable lives in `.env` at the project root. Copy the template first:

```
Copy-Item .env.example .env
```

| Variable | Purpose | Default |
|---|---|---|
| `CSN_SMTP_HOST` | SMTP server | — |
| `CSN_SMTP_PORT` | SMTP port | `587` |
| `CSN_SMTP_USER` | SMTP username | — |
| `CSN_SMTP_PASS` | SMTP password or app password | — |
| `CSN_SMTP_FROM` | envelope sender | the SMTP user |
| `CSN_SMTP_USE_TLS` | STARTTLS | `1` |
| `CSN_NOTIFY_OFFICIALS` | comma-separated recipients for new-FIR alerts | — |

Port and host have defaults, so only credentials are required.

### Gmail setup

1. Enable 2-Step Verification on the account.
2. Generate an **App Password** at <https://myaccount.google.com/apppasswords>.

Gmail rejects the normal account password.

1. Put the 16-character app password in `CSN_SMTP_PASS`.

### Without SMTP

The app is fully functional. `backend/mailer.py` prints the message to stderr and the API response includes a `devLink` so the reset flow can be completed from the console. `devLink` disappears automatically once SMTP is configured — that is the signal it worked.

```
{ "ok": true, "via": "email", "devLink": "http://127.0.0.1:4000/?reset=…" }
```

> Mail is best-effort by design. A send failure never loses an FIR; the handler commits the record first and logs the error.

---

## 5. Database lifecycle

The database is a single file: `backend/crimesight.db`. It is **created automatically** on first start — there is no install step for it.

At startup, in order:

1. `database/schema.sql` runs (idempotent — all statements are `CREATE … IF NOT EXISTS`)
2. missing columns are added by migration
3. `database/seed.sql` runs **only if** `police_stations` **is empty**
4. `database/south_india.sql` runs **only while the station table still holds the 25-row demo seed**

Step 4's floor of 1,000 rows is why restarting never re-imports 6,006 stations over your data.

### Start clean

```
Remove-Item backend\crimesight.db
python backend\app.py
```

Both seed files re-run automatically. This destroys all local changes.

### Backup

```
Copy-Item backend\crimesight.db backend\crimesight.db.bak
```

Or use SQLite's online backup, which is safe while the server is running:

```
python -c "import sqlite3; s=sqlite3.connect('backend/crimesight.db'); d=sqlite3.connect('backup.db');
s.backup(d)"
```

---

## 6. Re-importing datasets

```
python imports\import_datasets.py          # rewrite database/south_india.sql
python imports\import_datasets.py --apply  # …and load it
```

The importer parses `.xlsx` with the standard library alone — a workbook is a ZIP of XML, so no pandas or openpyxl is needed. 5,981 rows parse in a couple of seconds.

Re-run after editing either file in `data/`. `--apply` only loads when the station table is still the demo seed, so it cannot silently overwrite real data.

---

## 7. Tests

```
python tests\test_api.py
```

151 assertions. The suite builds a **throwaway database** in a temporary directory and never touches `backend/crimesight.db`, so it is safe to run against a live install.

Coverage: routing, signup and login, password hashing, the full permission matrix for all three roles, password reset including expiry and single-use enforcement, and the relationship graph.

The Public/admin boundary is pinned deliberately — widening `add` to include Public is the most likely regression, and this is what catches it.

---

## 8. Troubleshooting

| Symptom | Cause | Fix |
|---|---|---|
| `ModuleNotFoundError: flask` | dependencies not installed | `pip install -r backend\requirements.txt` |
| `Address already in use` | previous instance still running | kill it, or change the port in `main()` |
| `sqlite3.OperationalError: no such column: email` | a database predates the contact columns | delete the database and restart, or let the migration run |
| Page is blank, console shows a 404 for `three.min.js` | `frontend/vendor/` incomplete | restore it; the CDN fallback needs internet |
| Map renders but South India is flat | projection constants changed | `LON0=82.5`, `LAT0=22.5`, `SCALE=13` in the projection helper |
| 3D objects render black | `vertexColors: true` on geometry with no `color` attribute | drop the flag — instance colours arrive via `instanceColor` |
| Reset email not arriving | SMTP unconfigured or wrong credentials | check `CSN_SMTP_*`; confirm `devLink` is absent from the response |
| Reset says "account not found" | looked up by username | the route accepts **email or phone only** |
| Reset returns 403 | the account is police or admin | self-service reset is Public-only, by design |
| `handling` always shows "Direct" | source data | every seeded row has that literal value — a data property, not a bug |
| Login works but the page stays on the auth screen | JS error in console | check DevTools; usually a stale cached `index.html` |

---

## 9. Serving to other machines

The default bind is `127.0.0.1`, which is loopback-only and unreachable from another device. To expose it on the LAN, edit the `app.run()` call in `backend/app.py`:

```
app.run(host="0.0.0.0", port=4000, debug=False)
```

Then allow the port through Windows Firewall.

**Do this only on a trusted network.** The token scheme is self-asserting — see `SECURITY.md` — and traffic is plain HTTP. For anything beyond a lab, put a reverse proxy with TLS in front, switch to `waitress` or `gunicorn`, and replace the session scheme first.

---

## 10. What to do before calling this production-ready

1. Replace the self-asserting token with server-side sessions
2. Terminate TLS
3. Add rate limiting on the auth routes
4. Remove the seeded demo accounts
5. Serve through a real WSGI server
6. Add an audit log for writes and deletes
7. Configure SMTP and verify `devLink` no longer appears

The reasoning for each, and the gaps that remain, are in `SECURITY.md`.